CPU만 바꿨더니 2.4배 빨라졌다, SD 이득은 그대로

W05 추가 측정 · 양동균 · 2026-10-29 측정 · 지호 님 W05 3090 측정의 후속

> **한 줄 요약** — 지호 님 3090 측정을 **CPU만 Ryzen 5 5600X로 바꿔** 똑같이 돌렸더니, 8B 한 토큰 시간이 **69.5 ms → 28.8 ms**로 줄었다. GPU 계산 시간과 명령 수는 그대로였다 → **"CPU가 원인" 확정**. 다만 c는 0.75 근처에 머물렀고, SD는 12개 조건 전부 여전히 느리다 (0.56~0.90배).

1. 뭘 했나

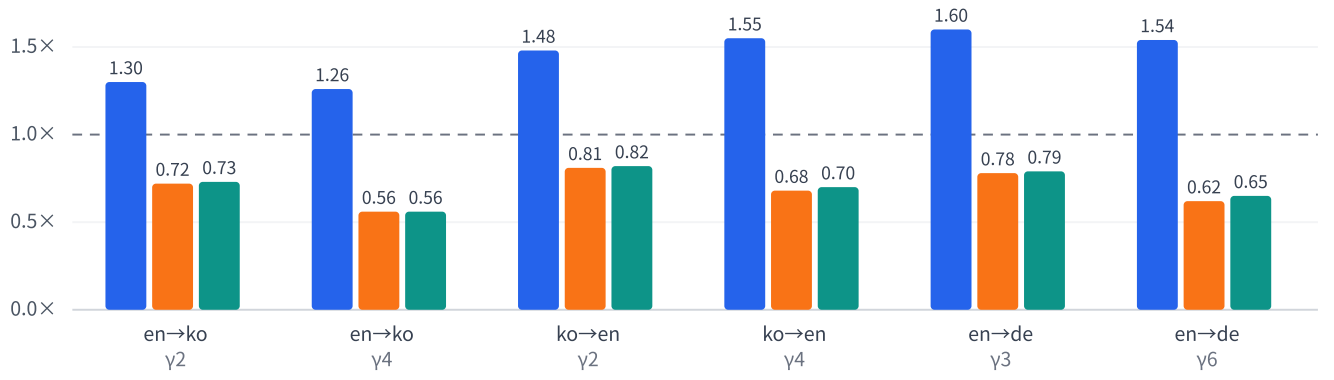
지호 님이 Xeon 서버의 3090으로 잰 실험을 그대로 다시 쟀다. 바꾼 건 CPU 하나뿐이다.

그대로 둔 것	바꾼 것
GPU (RTX 3090 24GB) · 모델 (target Qwen3-8B, draft 0.6B·1.7B, bf16) · FLORES 같은 12문장 · greedy · batch 1 · γ 값 · 스크립트 (`w05_3090/run_3090.sh`, `diag_3090.sh`) · 기준 커밋 b26acaa · torch 2.14.1 / transformers 5.18.0	**CPU**: Xeon E5-2673 v4 2.3 GHz (2016년 서버용) → **Ryzen 5 5600X** (Zen 3, 데스크톱용, 최대 4.6 GHz) vast.ai m:117733 (미국 미시간), On-Demand, $0.405/hr, 약 1시간 5분 · $0.44

서버 사양: Max CUDA 13.4, 드라이버 615.71, PCIe 4.0 x8, 6/12 CPU 공유. 판정 기준과 예측(P1~P4)은 측정 전에 적어 두었다.

2. 결과 — 토큰 시간은 절반 아래로, SD 이득은 그대로

■ M5 Pro (W04) ■ 3090 + Xeon (지호 님) ■ 3090 + 5600X (이번) · draft 0.6B 기준, 숫자 = SD 쓴 속도 ÷ 안 쓴 속도 · 점선 1.0 아래면 SD가 손해



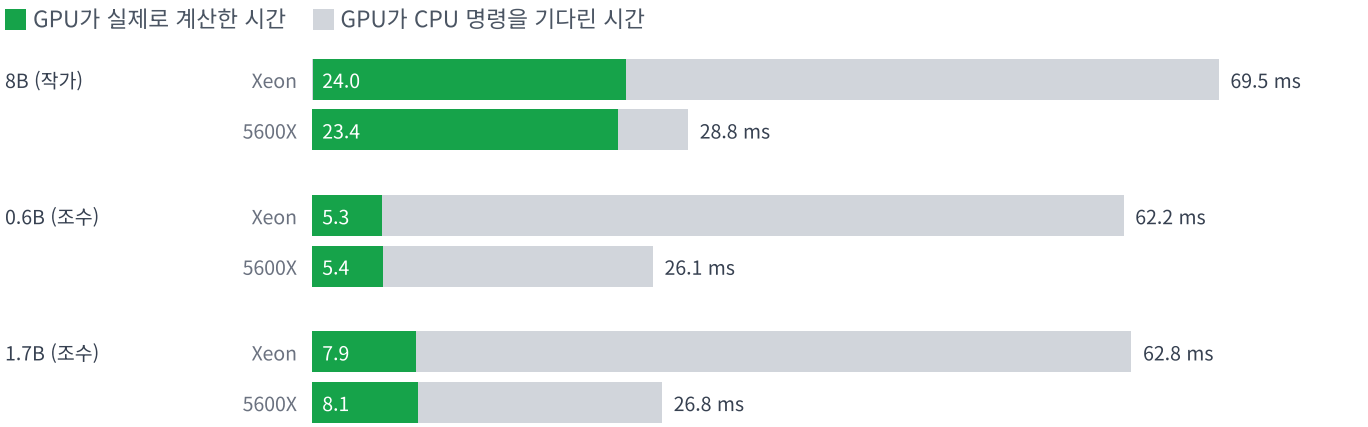
1.7B draft도 같은 모양 (Xeon 0.68~0.90배 → 5600X 0.70~0.90배). 수락률(α, τ)은 Xeon 측정과 12개 조건 모두 소수 셋째 자리까지 같았다 (greedy라 출력이 같다).

	3090 + Xeon	3090 + 5600X
8B가 한 토큰 쓰는 시간	69.5 ms	**28.8 ms (2.4배 빠름)**
c = 조수(0.6B) 1토큰 시간 ÷ 작가(8B) 1토큰 시간	0.75	**0.60~0.78** (안정 측정만 0.75~0.78)
SD speedup (12개 조건)	0.56~0.90	0.56~0.90

CPU가 빨라지자 작가와 조수가 **똑같은 비율로** 빨라졌다. c와 수락률이 그대로이니 SD 이득도 그대로다. SD 이득은 절대 속도가 아니라 **비율(c, r)**로 정해진다.

3. 왜? — 기다리는 시간만 사라졌다

진단 스크립트로 한 토큰을 만드는 동안 GPU가 실제로 계산한 시간과 CPU가 보낸 명령 수를 쟀다 (지호 님과 같은 `diag_cpu.py`).



모델	층 수	명령 수	GPU 실제 계산 Xeon → 5600X	한 토큰 시간 (진단) Xeon → 5600X
Qwen3-8B (작가)	36	2,038	24.0 → 23.4 ms	69.5 → **28.8 ms**
Qwen3-0.6B (조수)	28	1,590	5.3 → 5.4 ms	62.2 → **26.1 ms**
Qwen3-1.7B (조수)	28	1,590	7.9 → 8.1 ms	62.8 → **26.8 ms**

진단은 GPU 사용률을 같이 기록하느라 조수 쪽이 느리게 나온다 (진단 c = 0.91). c는 별도 비용 측정값을 쓴다.

> **비유: 주방장과 웨이터 (지호 님 비유의 후속)**
> 웨이터(CPU)를 빠른 사람으로 바꿨더니 주문서 한 장 건네는 시간이 약 34 µs에서 약 14 µs로 줄었다. 주방장(GPU)이 요리하는 시간과 주문서 장수는 그대로다.
> 그런데 주문서 장수는 여전히 코스 수(층 수)에 비례하고, 조수 요리(0.6B)는 주방장이 5 ms면 끝낸다. 웨이터가 2.4배 빨라져도 조수 쪽은 여전히 웨이터를 기다린다. 그래서 c가 그대로다.

숫자로 확인

· 명령당 CPU 시간: Xeon 69.5 ms ÷ 2,038 ≈ **34 µs** → 5600X 28.4 ms ÷ 2,038 ≈ **14 µs**
· 한 토큰 시간 ≈ max(GPU 계산, 명령 수 × 14 µs) → 8B = max(23.4, 28.5) ≈ 28.5 ms, 0.6B = max(5.4, 22.3) ≈ 22.3 ms → **c ≈ 0.78** (실측 0.75~0.78)
· c가 0.22로 내려가려면 명령당 시간이 약 3.4 µs 아래여야 한다 (0.6B가 GPU 계산 5.4 ms에 닿는 지점). CPU 교체만으로는 어렵다.
· 스레드 수를 6 → 13 → 4 → 1로 바꿔도 8B 28.2~29.0 ms → 지호 님 결론과 같다. **코어 하나의 속도** 문제다.
· 8B는 이제 GPU가 81% 바쁘다 (Xeon 35%). CPU를 더 빠르게 해도 8B는 23 ms 아래로 내려가지 않는다.

4. 그래서 알게 된 것

- **"CPU가 원인" 확정.** GPU 계산 시간(24.0 → 23.4 ms)과 명령 수(2,038)는 그대로인데, 기다린 시간만 45.5 ms → 5.4 ms로 줄었다. 바뀐 건 CPU뿐이다.
- **CPU를 바꿔도 c는 안 내려간다.** 작가와 조수가 둘 다 명령 전달에 묶여 있어서 같이 빨라진다. c를 0.22로 내리려면 CPU가 아니라 **명령 전달 방식**(CUDA Graph, llama.cpp)을 바꿔야 한다.
- **하드웨어(CPU)는 수락률이 아니라 속도만 바꾼다.** α, τ는 Xeon과 같고, speedup도 거의 같다.

측정 전에 적어 둔 예측	결과
P1. 8B 한 토큰 시간이 69.5 ms → 35~45 ms로 준다	**맞음** — 예상보다 더 줄어 28.8 ms
P2. c는 0.7~0.8에 머문다 (층 수 비 0.78)	**맞음** — 안정 측정 0.75~0.78, 흔들린 측정 포함 0.60~0.78. 어느 값이든 0.22와 멀다
P3. 0.6B와 1.7B의 시간이 계속 거의 같다	**판정 보류** — 안정 측정 2회에선 1.7B가 약 10% 느렸지만, 흔들린 측정 2회에선 차이가 없거나 반대
P4. SD는 12개 조건 모두 여전히 1배 아래	**맞음** — 0.56~0.90배

PDF 6절의 원래 기준("c가 0.22 쪽으로 내려가면 확정")은 측정 전에 "8B 토큰 시간이 CPU 속도에 비례해 주는지"로 바꿨다. 원래 기준이었다면 CPU가 원인인데도 "확정 안 됨"으로 읽혔을 것이다.

5. 조심할 점

비용 측정 4번 중 2번이 흔들렸다. 팀 기준(같은 측정 앞뒤 차이 10% 이하)을 넘었고, 원인은 확인하지 못했다. 인스턴스를 삭제해서 다시 재지 않았다.

회차	문맥	8B 앞 / 뒤 (ms)	앞뒤 차이	c 0.6B	c 1.7B	기준
1차	ctx_en	28.4 / 29.9	5.1%	0.747	0.845	**통과**
1차	ctx_ko	34.6 / 28.9	16.4%	0.666	0.635	**초과**
2차	ctx_en	38.0 / 28.7	24.4%	0.597	0.610	**초과**
2차	ctx_ko	28.9 / 28.9	0.1%	0.780	0.835	**통과**

- 흔들린 두 번 모두 **첫 측정이 느렸다** (34.6, 38.0 ms). 기준선이 느리면 c가 작게 나오므로, 흔들린 회차의 c(0.60~0.67)는 실제보다 낮게 나왔을 가능성이 크다.
- 두 회차가 각각 한 문맥씩 흔들려, 1차 값을 넣으면 ko 원문 4개 조건이 0.10~0.22, 2차 값(저장소 `summary_3090.md`)을 넣으면 en 원문 8개 조건이 0.15~0.26 실측보다 높게 예측된다. 기준을 통과한 문맥만 조합하면 12개 조건 모두 95% CI 안(최대 차이 0.055)이지만 측정 뒤에 고른 값이다 (`results/lang_3090_5600x/README.md`). **"팀 비용 공식이 맞는다"는 이번 측정만으로는 주장하지 않는다.**
- SD 측정 블록 앞뒤 속도 차이(드리프트)가 최대 9.4%로, Xeon 측정(최대 6.1%)보다 컸다.
- 결론에 주는 영향: c는 4번 모두 0.22와 멀고, 8B 토큰 시간은 진단(스레드 4종, 28.2~29.0 ms)으로도 따로 확인되므로 **"CPU가 원인"은 바뀌지 않는다.** 바뀌는 건 c의 정확한 값과 P3 판정이다.
- 서버 1대, 1회 측정. CPU 공유 매물(6/12)이었다.

6. 다음에 할 것

- **명령 전달을 줄인 실행 방식** (CUDA Graph 또는 llama.cpp CUDA)으로 같은 3090에서 다시 재기. 이번 결과로 c를 0.22 쪽으로 내릴 수 있는 건 이 방법뿐이라는 근거가 생겼다. 0.6B가 GPU 계산(5.4 ms)에 닿으면 c ≈ 0.23이 예상된다.
- 필요하면 5600X급 서버에서 비용 측정(`lang.py costs`, 약 5분)만 2~3회 반복해 c와 P3을 확정.
- GPU 클럭 조절 (공지 5번 Compute / Memory sweep) — 지호 님 계획 그대로.

원자료: `spec-decode/results/lang_3090_5600x/` (29개 파일, 지호 님 `lang_3090`과 같은 구성) · 재현: `spec-decode/w05_3090/README.md` (브랜치 w05-3090-measurements) · 예측·판정 기록: Claude Docs "W05 빠른 CPU 3090 측정 — 측정 전 예측·체크리스트"